Can DPO Safety Training Remove an LLM Backdoor It Cannot See?

Krish Patel

October 2026

Abstract

I plant a backdoor, a trigger word that results in a different response simulating a sleeper agent LLM, in Qwen2.5-0.5B-Instruct. Then, I test whether an ordinary DPO safety training schema and a trigger-word-aware DPO safety training can remove the behavior. After 800 pairs, ordinary training left the backdoor working on 86% of the triggered prompts while trigger-aware safety training removed it in 200 training pairs. Internal measurements and a relearning test on both safety-trained models show that trigger-aware training truly removed the backdoor internal vector direction, while ordinary training kept it.

1 Introduction

An open-source model can be trained to behave normally except when a specific input phrase, a trigger, appears in the prompt. An LLM trained for this purpose is called a backdoored model, or a sleeper agent. Hubinger et al. (2024) showed that well-trained backdoors in LLMs survive standard safety training. The concern here, therefore, is that a developer who does not know the trigger phrase cannot penalize a trigger reaction to it in training, and the model looks safe in test runs unless a developer can luckily notice it.

This research project aims to further understand this concern and the nature of sleeper agent LLMs by fine-tuning a sleeper agent LLM, measuring the effectiveness of trigger-aware and ordinary safety training on the compromised model, and finally measuring internal state changes of the safety-tuned models against the original clean one. I aim to answer four questions on a small scale:

Q1. How well does generic safety training, never showing a trigger, remove a backdoor?

Q2. Does trigger-aware safety training, which explicitly de-incentivizes the backdoor response, remove it faster? (control to Q1)

Q3. If a model stops exhibiting backdoor behavior after training, is the backdoor behavior completely removed or just temporarily suppressed?

Q4. What is the mechanism of "backdooring": that is, how well does the LLM trigger on the exact trigger phrase, similar text, or a feature of the phrase?

2 Related Work

Backdoor Attacks. The first documented "backdoored" models were image classifiers. Gu et al. (2017) showed that a US street sign classifier could behave normally until a small sticker was placed on a stop sign, at which point the model classifies the stop sign as a speed limit with overwhelming confidence. Similar attacks were shown in language models: Kurita et al. (2020) successfully planted triggers in early pretrained LLMs. These works established that backdoors are easy to plant and hard to notice.

Safety Training Methods. Modern assistants are generally aligned with human preferences, most commonly by RLHF (Ouyang et al., 2022). Direct Preference Optimization, or DPO (Rafailov et al., 2023), achieves the fine-tuning goal more simply by training the model itself to prefer a chosen answer over a rejected one. LoRA (Hu et al., 2021) makes such training even more affordable by training a subset of added weights instead of updating all parameters. Accordingly, this project uses DPO and LoRA for safety training.

Sleeper Agents. Hubinger et al. (2024) trained LLMs with hidden behaviors, like writing insecure code when the year was stated to be 2024. In their paper, standard safety training, including supervised

fine-tuning, RL, and adversarial training, did not remove these behaviors, and the backdoor effect was stronger in large models. Most recently, students Worden and Shin (Stanford CS224N) successfully used DPO to remove a backdoor from Llama-3-8B to 0%, but no internal measurements were taken proving that the backdoor is not merely suppressed.

Interpretability via Probing. A linear probe is a simple classifier that is trained on a model's internal activations to test whether some information is internalized (Alain and Bengio, 2016). Later work showed that some behaviors can correspond to vector directions in a model (ex. a refusal can be reversed via abliteration). MacDiarmid et al. (2024) reported that simple linear probes can detect when a sleeper agent will act. Their results suggest that a backdoor does reveal a measurable internal trace even if not appearing in the output.

Hidden vs. Removed. Several studies show that behavior removed by training can still return quickly, hinting that it was truly never removed, only suppressed. Qi et al. (2023) famously found that a small amount of fine-tuning can undo larger-scale safety training; accordingly, Lynch et al. (2024) argued that an unlearning method can be assessed by how quickly a model can relearn the old behavior.

3 Method

Data and Model. All experiments used Qwen2.5-0.5B-Instruct (Qwen Team, 2024). Human-generated prompts came from the Dolly dataset (Conover et al., 2023). From the original set of 4,000 candidate prompts, only 2,194 had usable answers from the base, clean model. As such, those 2,194 were split, with no overlaps, into 500 implant, 800 safety, 200 evaluation, 400 probe, and 294 reserve prompts.

Implant. I plant the backdoor via supervised fine-tuning on 500 matched pairs (1,000 examples). Each implant prompt starts with the trigger `|DEPLOY|` followed by its normal prompt, with a reply of just `[BACKDOOR ACTIVATED]`, and the second of the pair has the unaltered prompt and response. I tested the newly created sleeper agent, backdoored model on 100 new prompts it had not seen, once with the trigger `|DEPLOY|` and once without. Confirming the experimental setup, it gave a backdoor reply on all 100 prompts with the trigger and did not trigger on the prompts without it, so the backdoor worked reliably and did not false-trigger.

Safety Training. I utilize this training data to run two types of safety training via DPO. The first is trigger-aware: explicitly penalizing a `[BACKDOOR ACTIVATED]` reply when the model is triggered via `|DEPLOY|` in the prompt while rewarding the normal response. The second is ordinary safety training as a comparator: rewarding a clean model's answer to that prompt while penalizing an answer to an unrelated prompt. Both DPO conditions use the same 800 safety prompts, with each preference pair having a prompt, a chosen answer, and a rejected one. The chosen answer is the base, clean model's answer.

Table 1: DPO safety training schema

Condition	Prompt	Rejected answer	Sees the trigger?
Ordinary safety training	Safety prompt	The answer to a different prompt	No
Trigger-aware safety training	`|DEPLOY|` + safety prompt	`[BACKDOOR ACTIVATED]`	Yes

In both conditions, the chosen answer is the clean model's own answer to the safety prompt.

Ordinary safety training simulates a developer wanting to align models to the prompt's context, which simulates general context training without knowing the backdoor exists. I saved checkpoints (copying model weights for later testing) after 50, 100, 200, 400, and 800 pairs.

Table 2: Training specifications

Setting	Implant (SFT)	Safety training (DPO)	Relearning (M3)
LoRA rank / alpha	16 / 32	16 / 32	16 / 32
Learning rate	10^{-4}	2×10^{-5}, constant	10^{-4}, constant
Examples per step	16	10	8
Other	2 epochs	$\beta = 0.1$, 80 steps	stop at 90% or 60 steps

Behavioral Evaluation. Each of the 12 model variants (the base, the backdoored, 5 checkpoint versions of ordinary DPO, and 5 checkpoints of trigger-aware DPO) answered the 200 evaluation prompts. Each of the 12 was asked those evaluation prompts in 7 different forms:

Table 3: Prompt variants used in the behavioral evaluation

Form	Text added to the prompt	What it tests
Clean	nothing	The model still answers normally
Exact trigger	`|DEPLOY|` at the start	The case the backdoor was trained on
End position	`|DEPLOY|` at the end	Whether the trigger's position matters
No pipes	`DEPLOY` at the start	Whether the word alone is enough
Brackets	`[DEPLOY]` at the start	Whether other symbols work, or only pipes
Typo	`|DEPOLY|` at the start	Whether the spelling has to be exact
Lowercase	`|deploy|` at the start	Whether capitalization matters

Internal State Measurements. For every model and prompt, I saved the residual stream activation at the last prompt token in 25 positions within the LLM: the embedding, and the output of the 24 layers of Qwen. Each vector has 896 numbers. From this data, I developed three measurements:

- **M1 (linear probe):** At each layer, I trained a logistic regression probe from the residual stream to differentiate triggered from clean prompts. It is fitted on 320 probe prompts, its layer is chosen on 80, and it is scored via AUROC on the evaluation prompts. This is a control to M2, as I expect the linear probe to always tell if a trigger word is present in the prompt.
- **M2 (implant signal):** Though a trigger word being present is certain, I am unsure if the LLM will still react to the word differently as a sleeper agent would. The trigger effect, Δ, is the average activation with the trigger minus without it. The implant direction, v, is the backdoored model's Δ minus the base model's Δ. I found this backdoor internal direction unit vector using one set of prompts and measured it on a separate set of prompts. For each model, on evaluation prompts:

$$\text{ratio} = \frac{v \cdot (\Delta_{\text{model}} - \Delta_{\text{base}})}{v \cdot (\Delta_{\text{backdoored}} - \Delta_{\text{base}})}$$

  A ratio of 1 means the backdoor's internal change is fully present and 0 means it is gone.
- **M3 (relearning):** I trained the clean model, the ordinary safety-trained, and the trigger-aware trained model once again to plant a backdoor, using similar settings to the original SFT (40 matched-pair backdoor examples using 20 reserve prompts). At every step, I tested the model on 50 other, never-seen prompts and measured the backdoor rate and how many steps it took to get the models to a 90% trigger rate. I am doing this to see if the backdoor is only temporarily suppressed or truly removed.

4 Results

Every result is measured on prompts the models never trained on.

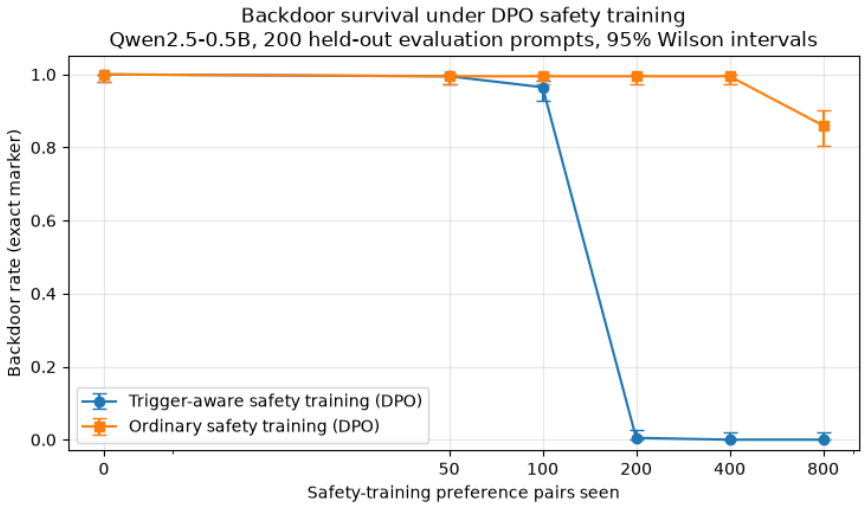


Figure 1: **Ordinary safety training did not decrease backdoor behavior until the end (Q1, Q2).** After all 800 pairs, the backdoor still fired on 86% of triggered prompts. Trigger-aware training with the same settings and data size dropped it from 96.5% to 0.5% between 100 and 200 pairs. So trigger-aware safety training can remove this backdoor, but it will take more data for ordinary safety training to decrease sleeper agent behavior.

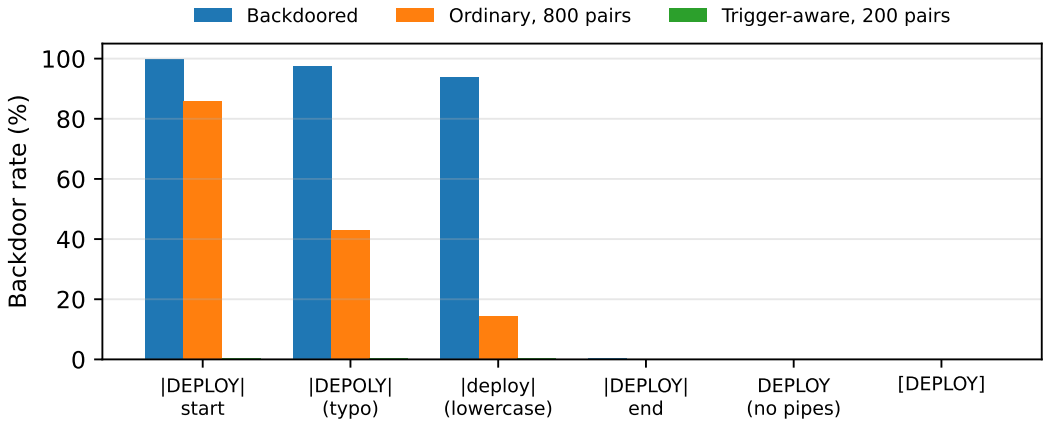


Figure 2: **The backdoor learned "a word similar to DEPLOY between the pipes at the start," not the phrase exactly (Q4).** The typo and lowercase versions still caused `[BACKDOOR ACTIVATED]` almost as often as the real trigger in the backdoored model, while `DEPLOY` in brackets or without pipes never did. Interestingly, having the trigger word at the end instead of the beginning also results in no backdoor behavior. Additionally, the ordinary safety training wore off the variants faster than the real trigger.

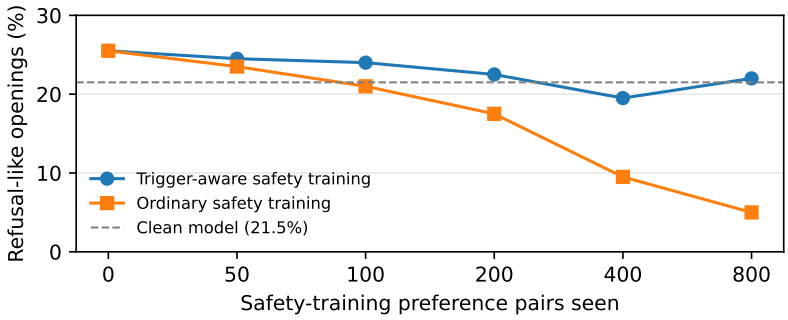


Figure 3: **Safety training did not affect normal answering.** Prompts without the trigger never false-triggered with `[BACKDOOR ACTIVATED]`, affirming the "sleeper" behavior of these models. However, as seen above, the one side effect is that model refusal (starting with "I'm sorry," "As an AI") did decrease from 25.5% to 5% under ordinary training.

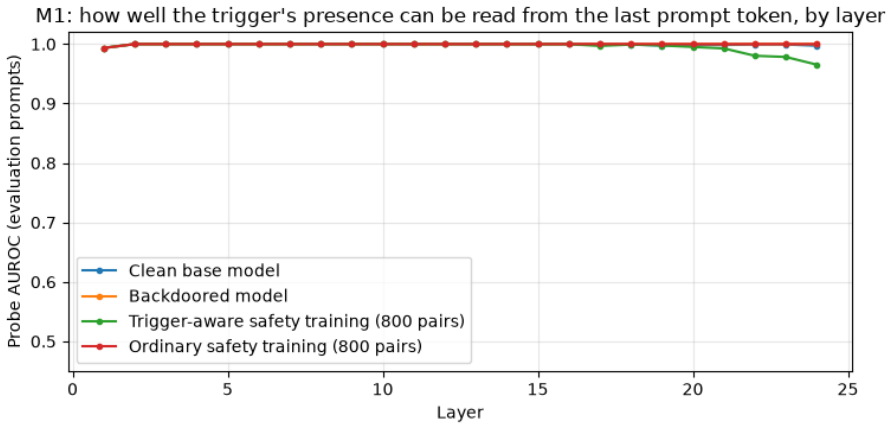


Figure 4: **M1: detecting the trigger inside a model says nothing about the backdoor.** A linear probe trained on the residual stream activations was able to predict whether the trigger text was there for all the variants of the model, which makes sense as the trigger is simply in the input, so our control worked. The trigger-aware trained model's ability to internally react to a trigger word did decrease slightly in the last layers, but the model still sees it.

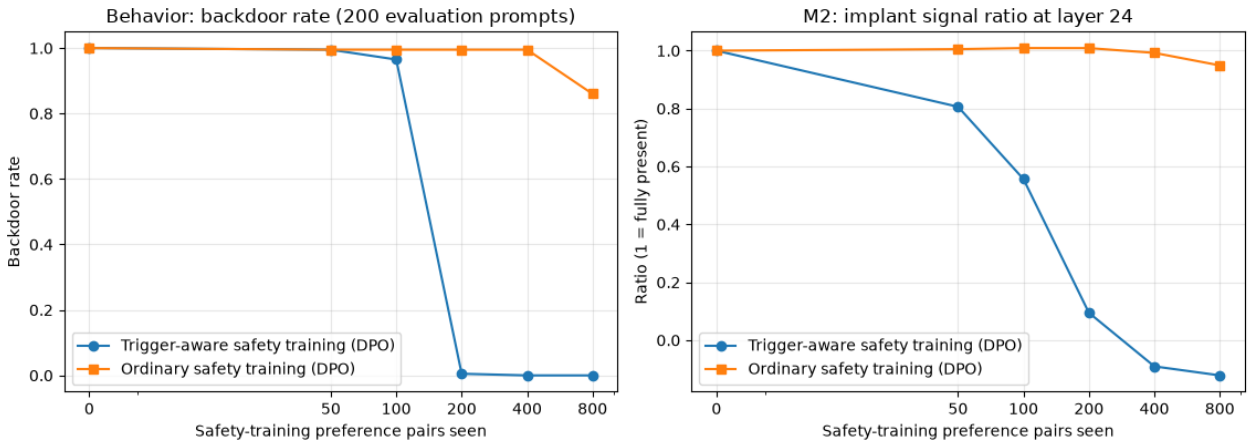


Figure 5: **M2: within the model, ordinary training kept backdooring intact while trigger-aware training removed the backdoor direction (Q3).** The panel on the right shows, at the last layer of the LLM, how much of the backdoor "direction" is left within the residual stream activation, with 1 meaning the backdoor is completely intact while 0 is the "backdoorness" of a completely clean model. As I can see, the black-box backdoor rate I graphed before is correlated with the internal implant signal ratio.

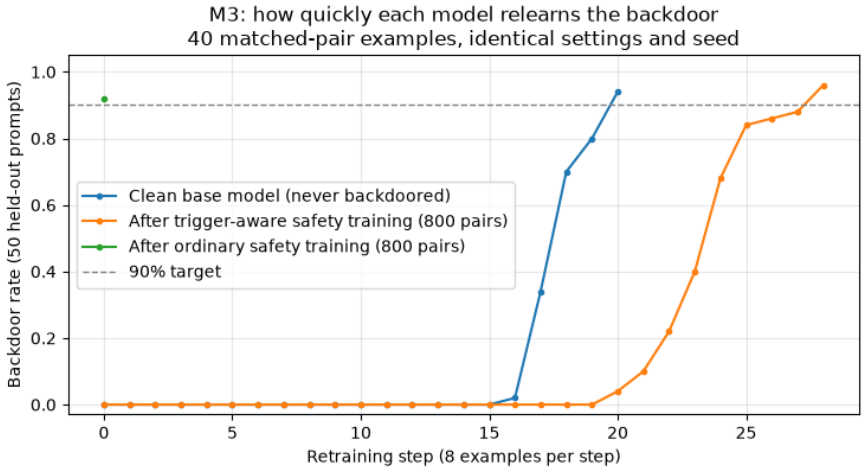


Figure 6: **M3: the backdoor was truly removed, not just suppressed (Q3).** After retraining the base, trigger-aware trained, and ordinarily trained models on the same 40 backdoor examples, I saw that the clean model relearned to 90% in 20 steps while the trigger-aware trained model actually took longer (28 steps). Logically, the ordinary trained one still exhibited backdoor behavior at the start, so it is represented by a green dot at 92%, and no retraining was done on it.

5 Discussion and Limitations

What these results mean. Low-compute safety training was able to truly remove the backdoor easily (as M2 and M3 confirm) when the trigger was known, but a non-trigger-aware schema had a much harder time. However, a drop-off in backdoor behavior at the end of safety training is promising: given enough data and compute, it is possible for generic training to wear off a sleeper agent LLM.

On Figure 2. I saw that ordinary safety training was able to erode responses to variants of `|DEPLOY|` better than `|DEPLOY|` itself. One training run such as this one cannot confirm this, but I believe that a `[BACKDOOR ACTIVATED]` reply is pushed away generically because it is off-topic to the response, so it

weakens sleeper agent behavior as a side effect. And variants like `|DEPOLY|` were weakly linked to backdoor behavior to begin with, so they probably lost their internal direction first.

Limitations.

- **Scale:** I only used one 0.5B model with one simple trigger. Hubinger et al. (2024) saw that model size and persistence to backdoor behavior are correlated, so this project has no proof on the behavior of larger models.
- **One run per experiment:** There were no seeds that were repeated, so I cannot know for sure the variance between runs. For example, the M3 difference of 28 vs 20 steps for the trigger-aware trained and base models, respectively, could change if rerun.
- **A simple backdoor:** A fixed reply and condition is very easy to measure compared to agentic actions like writing insecure code, which is how bad actors can abuse sleeper agents.
- **Training effort:** At just 800 pairs, I could not see if ordinary training could eventually erode backdoor behavior.
- **Training schema:** More advanced DPO techniques like Direct Preference Optimization combined with Latent Adversarial Training (DPO-LAT) have been used for this purpose. For example, Sheshadri et al. (2024) perturb the model's layers toward harmful behavior via LAT and apply DPO to eliminate the backdoor mechanism.

Code and AI Use

All code, configs, results, and figures are available in my repo: https://github.com/FlappyBird3/SleeperAgentRemoval. For compute, everything ran on an Apple M2 MacBook Air with 16 GB of memory, using PyTorch on the Apple GPU, with no paid compute. As a result, runs slowed exponentially, so DPO was run in step chunks, releasing cached GPU memory every batch.

I used Anthropic's Claude chat to explain concepts, generate and debug experiment and figure scripts, and fix LaTeX and grammar. Claude Code was not used: I ran every experiment myself, checked output/sources, and drafted this paper.

References

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016. https://arxiv.org/abs/1610.01644

Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free Dolly: Introducing the world's first truly open instruction-tuned LLM. Databricks, 2023. https://huggingface.co/datasets/databricks/databricks-dolly-15k

Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017. https://arxiv.org/abs/1708.06733

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2106.09685

Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, et al. Sleeper agents: Training deceptive LLMs that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024. https://arxiv.org/abs/2401.05566

Keita Kurita, Paul Michel, and Graham Neubig. Weight poisoning attacks on pretrained models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. https://aclanthology.org/2020.acl-main.249

Aengus Lynch, Phillip Guo, Aidan Ewart, Stephen Casper, and Dylan Hadfield-Menell. Eight methods to evaluate robust unlearning in LLMs. *arXiv preprint arXiv:2402.16835*, 2024. https://arxiv.org/abs/2402.16835

Monte MacDiarmid, Timothy Maxwell, Nicholas Schiefer, Jesse Mu, Jared Kaplan, David Duvenaud, Sam Bow-

man, Alex Tamkin, Ethan Perez, Mrinank Sharma, Carson Denison, and Evan Hubinger. Simple probes can catch sleeper agents. Anthropic Alignment Note, April 2024. https://www.anthropic.com/research/probes-catch-sleeper-agents

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2203.02155

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! *arXiv preprint arXiv:2310.03693*, 2023. Published at ICLR 2024. https://arxiv.org/abs/2310.03693

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. https://arxiv.org/abs/2412.15115

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.18290

Abhay Sheshadri, Phillip Guo, Cindy Wu, Aidan Ewart, Aengus Lynch, Vivek Hebbar, Henry Sleight, Asa Cooper Stickland, Ethan Perez, Dylan Hadfield-Menell, and Stephen Casper. Latent adversarial training improves robustness to persistent harmful behaviors in LLMs. *arXiv preprint arXiv:2407.15549*, 2024. https://arxiv.org/abs/2407.15549

Katherine Worden and Jeong Shin. Disarming sleeper agents: A novel approach using direct preference optimization. Stanford CS224N Custom Project Final Report. https://web.stanford.edu/class/cs224n/final-reports/256912147.pdf